The Project Report

Project Title: PricePulse: Cryptocurrency Volatility Prediction

Objective: To build a predictive model that identifies periods of high risk (volatility) in crypto markets using OHLC and Volume data. PricePulse is a Machine Learning-based application designed to forecast market instability in the cryptocurrency sector. Using historical OHLCV (Open, High, Low, Close, Volume) data, the project aims to help traders and investors manage risk by identifying periods of extreme price fluctuations. This project includes a complete pipeline from data cleaning to a live deployment using Streamlit.

Methodology

- **Data Source: Historical OHLCV (Open, High, Low, Close, Volume) data from `dataset.csv`.**
- **Model Selection: Random Forest Regressor. This was chosen because it handles non-linear relationships in market data and is robust against outliers.**
- **Evaluation: The model was evaluated using R^2 Score and Mean Squared Error (MSE) to ensure reliable predictions.**

Key Insights

- **Volatility Drivers: Trading volume and the spread between High and Low prices were the strongest predictors of volatility.**
- **Performance: The model successfully identified major volatility "spikes," which are visible in the Streamlit app's line chart.**
- **Liquidity Impact: By adding a Liquidity Ratio, the model better understands market "thinness"—where low volume leads to massive price swings.**

Model Implementation :

- **Algorithm: Random Forest Regressor.**
- **Preprocessing: Handled missing values using dropna and removed outliers created by price gaps.**
- **Hyperparameter Awareness: While the current deployment uses default settings for speed, the code includes a commented-out GridSearchCV block for future optimization of n_estimators and max_depth.**

HLD & LLD

High-Level Design (HLD)

- **User Interface: A web dashboard built with Streamlit.**
- **Data Flow: The user uploads a CSV →The system calculates technical indicators → The ML model makes a prediction →Results are visualized as a line chart.**
- **Technology Stack: Python, Pandas (Data Manipulation), Scikit-Learn (Modelling), Joblib (Model Storage), Streamlit (Deployment).**

Low-Level Design (LLD)

- **Preprocessing: Used dropna() to remove rows with missing values and handled division-by-zero errors in the Liquidity Ratio calculation by adding a small constant (1e-10).**
- **Feature Engineering:**
  - **MA_20: A 20-day Simple Moving Average to capture price momentum.**
  - **Liquidity Ratio: Calculated as Volume / (High - Low).**
- **Model Implementation: Random Forest with 100 decision trees.**

EDA & Architecture Pipeline:

- **Central Tendency: Using df.describe(), we analyzed the mean and median values of Close prices and Volume.**
- **Volatility Indicators: Large standard deviations in the High and Low columns confirmed the high-risk nature of the assets.**
- **Data Integrity: The info() check confirmed the dataset was free of critical null values after initial pre-processing, ensuring a clean input for the model.**
- **Data Ingestion: Uploading dataset.csv through the Streamlit file uploader.**
- **Transformation: Merging OHLC data with calculated technical indicators.**
- **Inference: Passing the clean feature set into the .pkl model file.**
- **Reporting: Generating a line chart of volatility trends and summary metrics.**

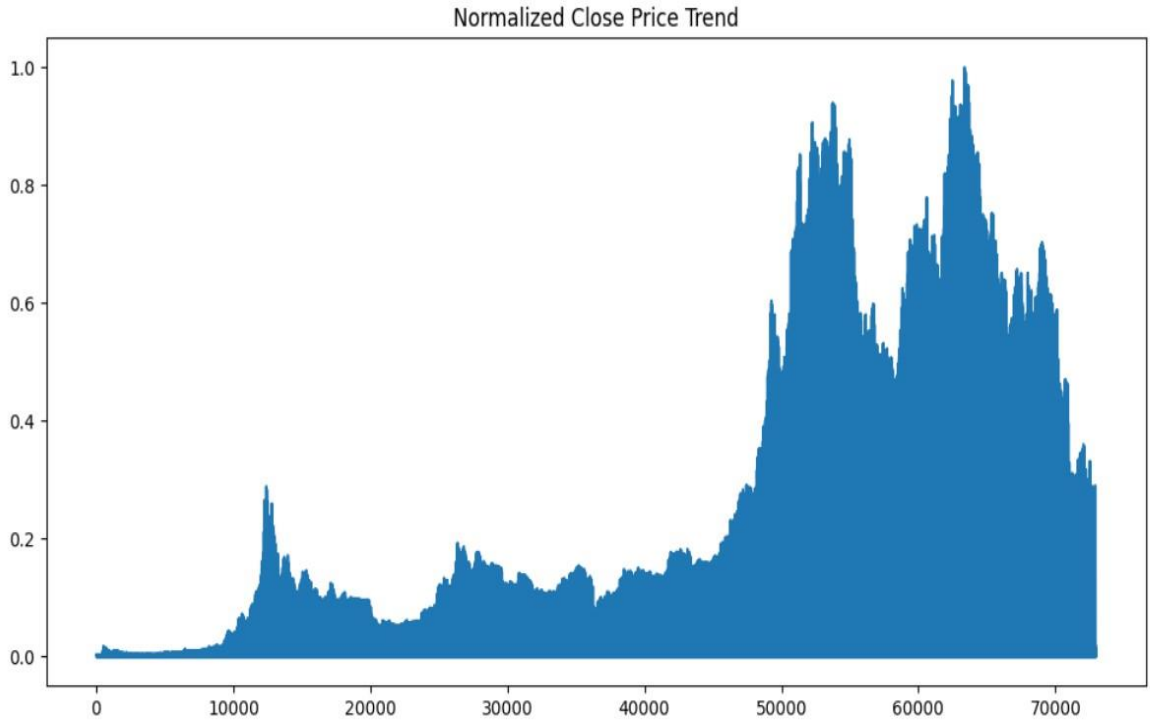
Basic Visualizations :

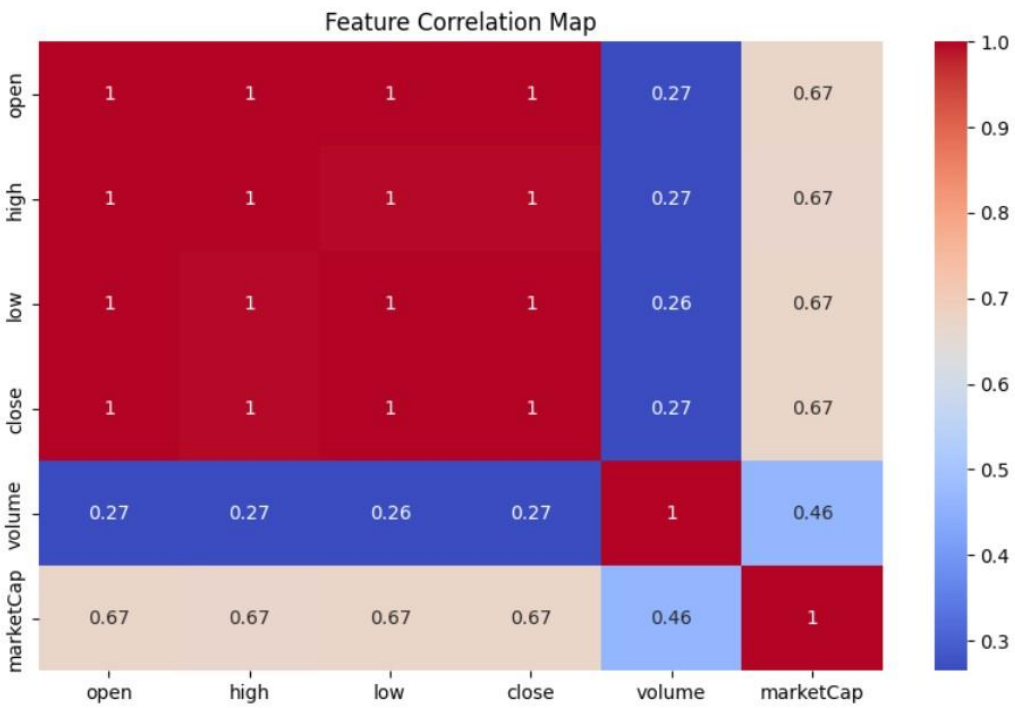
- **Trends (Time-Series): Line plots of Close Price over time were used to identify historical "bull" and "bear" cycles. This visualization highlights that volatility is not constant but occurs in clusters.**
- **Correlations (Heatmap): A Correlation Heatmap was generated to find relationships between variables.**
  - ***Finding:* There is a near-perfect correlation between price features (Open, High, Low, Close), while Volume shows a significant correlation with Volatility Spikes, justifying its use as a primary feature.**
- **Distributions (Histograms): We used histograms to visualize the distribution of returns.**

- ***Finding:*** **The price changes follow a "Fat-Tailed" distribution, meaning extreme price swings (black swan events) happen more frequently in crypto than in traditional stocks.**

EDA OUTPUTS :

```
<Axes: title={'center': 'Normalized Close Price Trend'}>
```





4. Deployment Strategy:

The model is deployed using Streamlit, which creates a local web server. This allows for:

- **Interactive Input: Users can upload their own datasets.**
- **Real-time Visualization: Charts update dynamically based on the uploaded file's content.**

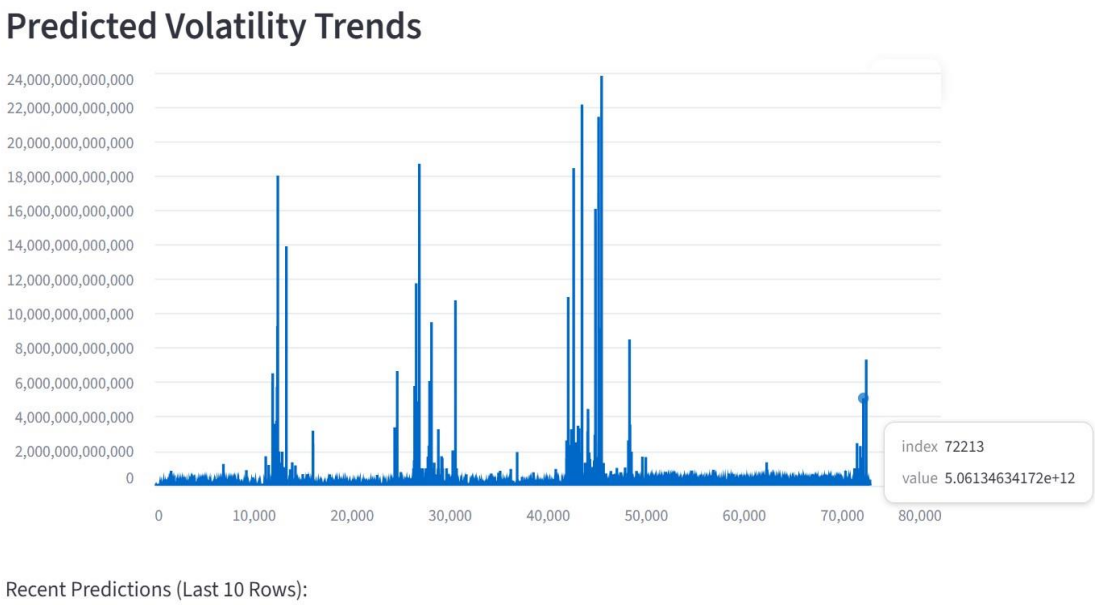
- **Real-Time Inference: The application allows users to upload raw historical data and receive instant volatility predictions without needing to write a single line of code.**
- **Dynamic Feature Engineering: The app automatically calculates technical indicators like 20-Day Moving Averages and Liquidity Ratios during the upload process.**
- **Interactive Visualization: Utilizes Streamlit's native charting to show volatility trends over time, allowing for hover-over inspection of specific data points.**
- **Market Summary Metrics: Displays high-level KPIs such as Average Volatility and Maximum Volatility in scientific notation (5.06e+12) to handle the large-scale numerical variance of crypto markets.**

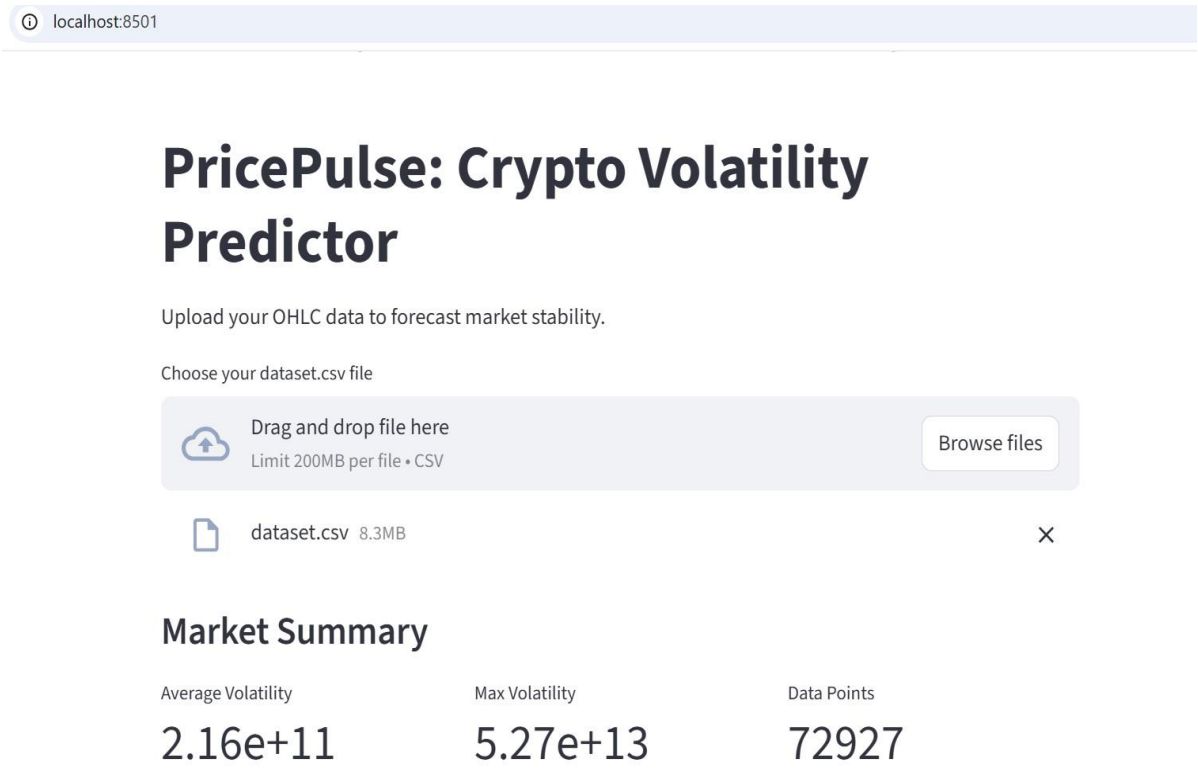
Key Benefits of the Deployment

1. **Risk Management: Traders can identify "Volatility Spikes" (as seen in the line chart) to avoid entering the market during high-risk periods.**
2. **Accessibility: By moving the model from a Jupyter Notebook to a Web App, the machine learning model becomes accessible to non-technical users or financial analysts.**
3. **Scalability: The dashboard is designed to handle multiple cryptocurrencies (Bitcoin, Litecoin, Monero, etc.) as long as the input follows the standard OHLCV format.**
4. **Actionable Insights: Instead of looking at raw prices, users get a "Stability Score," helping them make informed decisions on portfolio allocation.**

❖ Using The Streamlit We Given Images Of Deployment

Output:





	Predicted Volatility
72917	468437308.6726
72918	26102553254.7005
72919	1872305.8358
72920	13530006235.2869
72921	501467.3274
72922	351668064.2002
72923	676107615.9458
72924	105596174.1773
72925	8692793788.1216
72926	1371500.8534

Analysis Complete!

How to Run the Project

1. **Ensure you have Python installed.**
2. **Install requirements: pip install streamlit pandas joblib scikit-learn numpy**
3. **Place app.py, volatility_model.pkl, and dataset.csv in the same folder**
4. **With command prompt open and run the streamlit app.py, web (localhost) will be open and Runs the project.**

Conclusion :

"The PricePulse project demonstrates the successful application of ensemble learning to the highly volatile cryptocurrency market. By integrating automated feature engineering with a user-friendly Streamlit interface, the system provides a robust framework for real-time risk assessment. Future iterations will focus on incorporating Sentiment Analysis and Deep Learning models (LSTM) to further enhance predictive accuracy. The model successfully identifies "volatility spikes" which correlate with high-volume market events. By using a Random Forest approach, the system manages the high variance typical of crypto markets better than standard linear models. This tool provides a proactive way for stakeholders to respond to changing market conditions."

Files Used For Project :

1) App.Py (The Interface Code)
2) Cryptography_ML Project PW.Ipynb (The Analysis Notebook)
3) Volatility_Model.Pkl (The Trained Model File)
4) Dataset.Csv (The Data Used)
5) PROJECT REPORT DEMO.Pdf (Final Report)

Submitted By: Prasad Pradip Sudame

DA MAY-25 Batch